

Steps to Use a Neural Network for Prediction

1. ■ Prepare the Dataset - Collect data (CSV, images, text, etc.) - Split into features (X) and labels (y) - Convert to tensors - Normalize/standardize if needed - Split into train, validation, and test sets
2. ■ Create a DataLoader (optional but common) - Wrap data into TensorDataset or custom Dataset - Use DataLoader for batching and shuffling
3. ■ Define the Model (Architecture) - Decide input size, hidden layers, output size - Use nn.Module and nn.Linear layers - Add activations (ReLU, Sigmoid, Softmax)
4. ■■ Choose Loss Function - Regression → MSELoss - Binary classification → BCELoss / BCEWithLogitsLoss - Multi-class classification → CrossEntropyLoss
5. ■ Choose Optimizer - torch.optim.SGD, Adam, etc. - Set learning rate (e.g., lr=0.001)
6. ■ Training Loop For each epoch: - Forward pass → model(X) - Compute loss - Backward pass → loss.backward() - Optimizer step → optimizer.step() - Zero gradients → optimizer.zero_grad()
7. ■ Track Metrics - Accuracy (classification) or RMSE (regression) - Print logs or use TensorBoard
8. ■ Validation - After each epoch, evaluate on validation set - Use torch.no_grad() to avoid gradient calc
9. ■ Save/Load Model (optional) - Save: torch.save(model.state_dict(), 'model.pt') - Load: model.load_state_dict(torch.load('model.pt'))
10. ■ Make Predictions - Put model in eval mode: model.eval() - Pass new data through model - Post-process output (e.g., argmax for class)